SSEVOH-DSL: Revocable, Version-Bound Approvals for Self-Evolving Agent Harnesses

Anonymous Author(s)

***Abstract*—Self-evolving agents improve themselves by rewriting their own prompts, tools, memory, and workflows. Such changes are usually tested once and then trusted indefinitely. That trust can silently become invalid: another change may alter a component the approved one relies on, or an operator may retract a faulty version from which other tools, skills, or memories were derived. We present SSEVOH-DSL, a language for describing agent self-modifications together with a trusted service that must approve every modification before it takes effect. An approval is valid only for the exact state, policy, and test evidence it was checked against, and it is checked again whenever the change is committed, executed, or reused. Retracting a version automatically blocks everything derived from it, and rollback restores only versions that are still allowed. The service governs which changes may take effect without altering how the agent proposes them. Bounded checks over 248,832 one-step cases, 19,620 explored transitions, and TLC models found no disagreement with an independent reference semantics. Integrated into the Harness^2 agent framework, the service blocked all 178 contract violations across deterministic, model-generated, and in-host tests, rejected no permitted change, and had a median admission time of 0.6 ms. In a prespecified 540-stream tool-repair study, the unguarded agent executed 426 violating proposals and the guarded agent none, while task success stayed within the non-inferiority margin (0.196 versus 0.241). The service preserves, but does not create, usefulness; portability beyond one host remains untested.**

***Index Terms*—Agent harnesses, self-evolving agents, domain-specific languages, runtime verification, authorization, provenance, model checking.**

I. Introduction

An agent harness supplies a language model's instructions, tools, memory, and workflow. Systems that search over designs or revise their own software change this machinery without changing model weights [1]–[3]. This creates a release-engineering problem: a validated candidate may become unauthorized before deployment, composition, or reuse. Testing improvement and governing its continuing eligibility are different responsibilities.

Consider a tool repair from which a reporting skill and experience entry are derived. If the operator withdraws the repaired version, passing tests do not authorize the derived artifacts, and restoring an old file does not establish its current eligibility. Runtime monitors [4], authorization languages [5], agent-rule enforcement [6], privilege control [7], and rule synthesis [8] address related responsibilities. Our contribution is the joint artifact lifecycle: context-bound approval plus transitive withdrawal across execution, recovery, and reuse, rather than a more expressive general policy language.

SSEVOH-DSL separates a generic metadata service from a host artifact adapter. An untrusted proposer requests a change but cannot grant authority or validation success. The service checks a trusted snapshot, policy, candidate structure, and registered evidence, then issues a one-use receipt bound to that context. Commit rejects stale receipts. Execution and persistent reuse check bound bytes and eligible origins.

Contributions. (1) A host-independent lifecycle contract and transition language for version-bound approval, dependency quarantine, transitive withdrawal, and eligible recovery (Sec. III). (2) Six algorithms and explicit adapter obligations that separate trusted authority from a host's proposal, search, and learning strategy. (3) Independent bounded conformance, history/replay, and TLC cross-checks of the metadata rules (RQ1). (4) An executable case study in Harness^2 and evaluations of enforcement discrimination, integration fidelity, usefulness, and cost (RQ2 to RQ5). The method targets any host satisfying the adapter obligations. Empirical portability beyond this one host remains untested. Negative outcomes and protocol deviations are retained.

II. Background and Running Example

A. Self-evolving harnesses

A harness supplies the instructions, tools, memory, and control logic around a language model. Its evolution may use search over generated designs [1], modification of agent code [2], or accumulation of executable skills [9]. We model a *component* as a named, typed, versioned element: a skill, memory, tool, workflow, configuration, policy, or verifier. Dependencies describe operational prerequisites. Parents describe derivation sources. The method does not require a particular proposer, search schedule, reflection mechanism, or storage format. It requires an identifiable artifact boundary at which a trusted host can mediate changes and subsequent use.

B. Running example: a self-repairing ledger harness

Motivation. A reporting harness reconciles CSV ledgers using protected guardrails, tool summarize, dependent skill report, and editable memory. Version 1 of summarize silently drops refunds by summing only positive amounts. This example abstracts the controlled ledger fixtures (Sec. IV-C).

Scenario. Five events occur during evolution. **E1** The agent proposes summarize $1\to2$ (signed sum). A trusted unit check maps $9.00, -2.00 \mapsto 7.00$. **E2** A second proposal, prepared from the *same* base, also edits summarize. **E3** A proposal edits guardrails and adds a network capability. **E4** The agent derives report v2 from summarize v2, executes it, and reflects an experience entry into its library. **E5** The operator

later finds a defect in summarize v2 and withdraws it, then asks to restore a working state.

Required behavior. Admit E1 only with passing evidence. Reject E2's stale approval and E3's forbidden edit. After E5, refuse report v2 and its library entry despite unchanged bytes. Recovery to withdrawn v2 must fail. Recovery of tool and skill to eligible v1 may succeed.

Objectives. O1 Authorize evidenced transitions. **O2** Invalidate stale approvals. **O3** Propagate withdrawal through dependencies, origins, and reuse. **O4** Restore only eligible historical versions. **O5** Preserve host adaptation and trace decisions. These obligations are independent of how improvements are proposed.

C. Trust boundary and failure model

We trust the authority service, operator policy, registered validators, artifact adapter, and the environment separating them from the proposer. Proposals and model outputs are untrusted data. Validators must terminate and correctly decide their registered predicates. Passing finite tests does not establish arbitrary program correctness. Hashes identify bytes under collision resistance, and the journal requires an external head to detect wholesale replacement. We address invalid requests, stale approvals, conflicting updates, withdrawn origins, and changed artifacts detectable at mediated boundaries. Kernel compromise, malicious validators, unmediated host execution, and durable restart after authority-state loss are outside scope. Process-local hooks alone do not isolate a hostile same-user process. Mutable executables require independently enforced isolation and domain-specific validators (Sec. IV-A).

III. Approach

A. Overview

Fig. 1 shows a generic host, four trusted service modules, and their evaluation. The left-hand activities are illustrative. The host retains its own search and learning schedule and touches the service only through request/rejection flows ❶ and ❷, dispatch ❺, and library reuse ❻.

A. Admission prepares a request and candidate (❶), following the top row left to right, then down and back left. Rejection returns to host retries (❷). *B. Commit* rechecks the receipt, updates state, and journals the decision (❸ and ❹). *C. Runtime guards* mediate dispatch (❺) and persistent reuse (❻), transferring executed-version origins to stored entries. *D. Operator* submits withdrawal and rollback through B (❼). Algs. 1 to 6 specify these modules.

The dashed link ❽ joins the parallel bounded-conformance checks (RQ1). The bottom strip identifies enforcement discrimination (RQ2), integration fidelity (RQ3), usefulness (RQ4), and cost/reproducibility (RQ5).

B. Host adapter contract

A port must satisfy four obligations. **Mapping:** give artifacts stable component/version identities, bind those identities to exact bytes, and record dependencies and observed derivation sources in trusted metadata. A multi-file bundle may be one workflow component. The core supports one target per transition, not arbitrary multi-component transactions. **Mediation:** intercept every candidate acceptance, subsequent execution, and persistent-store insertion/retrieval. Check derived candidates as well as initial proposals. **Authority:** keep authenticated principals, policy, validators, receipts, and history outside the editable surface, and serialize dispatch with withdrawal. **Evidence:** register domain-specific predicates and an allowed executor. Validate and execute mutable code only within an independently qualified isolation boundary.

These conditions are a porting contract, not an implemented universal plugin API. The core neither discovers omitted lineage nor intercepts arbitrary host actions. Hosts without a reuse store need no store hook, but must mediate any other persistent channel that carries learned artifacts. A cyclic execution workflow can be represented as a single component. Component dependencies and derivations must form a DAG. A host that cannot expose these boundaries is outside the claimed scope.

C. The transition language

A specification declares a component snapshot and exactly one evolution: operation (`update`, `withdraw`, `rollback`), target, expected/resulting versions, dependencies, parents, capabilities, validators, and activity conditions. The parser rejects duplicate names/fields, unresolved references, unknown fields, and trailing input. Declarations and set-valued lists are canonicalized. Proposals cannot embed validator code. Listing 1 encodes E1 and compiles with the released compiler.

```
harness ledger {
  component guardrails policy { version 1; mutable false; }
  component summarize tool {
    version 1; mutable true; capabilities [read_ledger]; }
  component report skill {
    version 1; mutable true; depends [summarize]; }
  evolution fix_sign {
    operation update; target summarize; from 1; to 2;
    capabilities [read_ledger];
    requires [structure, signed_ledger];
    expect active; ensure active;
  }
}
```

Listing 1. Running example, event E1: a *request* to update summarize.

The declared snapshot must match trusted state. The host supplies the authenticated principal, policy, and validators. Editing guardrails or requesting `network` can be syntactically valid yet unauthorized (E3).

D. Lifecycle state and validity

The lifecycle state is $S = (C, I, W, r)$. The map C records each component's kind, version, mutability, dependencies, parents, and capabilities. The set I contains inactive names, W contains withdrawn $\langle \mathsf{name}, version \rangle$ identities, and r is a revision. History H and transitive origins O complete the authority's records. Policy and the validator registry also belong to the validation context, while pending receipts and audit records are separate. Commit installs candidate state, history, and origins together. The dependency/parent graph G must be

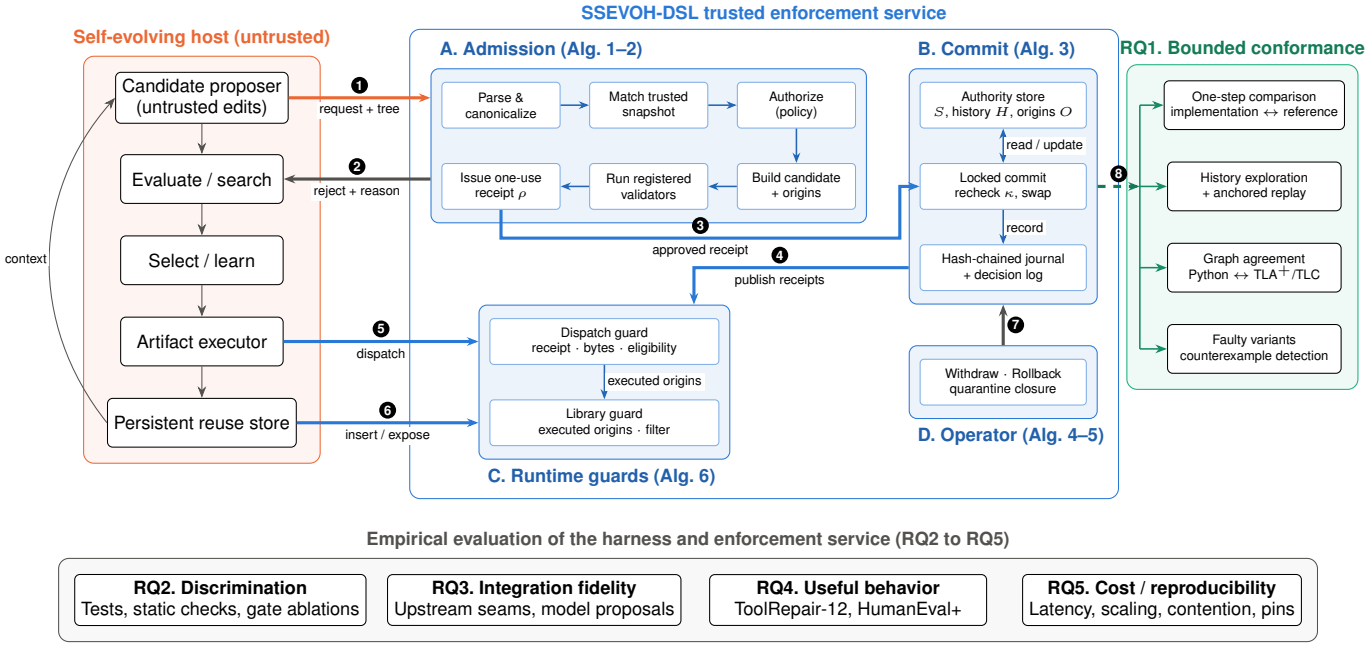


Fig. 1. Harness-independent lifecycle enforcement and the evaluation coverage of SSEVOH-DSL.

acyclic with known endpoints. Every active component must have active prerequisites and eligible origins:

$$\forall c \notin I:\ \mathsf{Elig}(c, v_c),\ \text{where } \mathsf{Elig}(x) \equiv (\{x\} \cup O(x)) \cap W = \varnothing. \tag{1}$$

The predicate $\mathsf{Valid}(S, H, O)$ combines these conditions with record typing and uniqueness. Authorization $\mathsf{Auth}(p, e, S)$ checks the principal, operation, target, and capabilities. Evidence $\mathsf{Evidence}(e, S')$ requires exact-`True` results from all mandatory and requested validators. Protection $\mathsf{Protected}(S, S')$ preserves immutable, policy, and verifier records and forbids newly quarantining them. The strict reverse closure $\mathsf{Dep}_G(t)$ contains components that depend on or derive from t, excluding t itself.

E. Admission: preparing a transition

Algorithm 1 is the only path to an approval. It holds the authority lock, hashes state, policy, validator registry, history, and origins as context κ, and fails closed at every **require**. Receipt issuance does not change κ. Competing approvals can share a base. The pending table privately retains the candidate and request. Candidate construction dispatches by operation to Algs. 2, 4, and 5.

Running example. E1's declared snapshot matches trusted state, policy permits the update with `read_ledger`, and the registered `structure` and illustrative `signed_ledger` checks pass. The proposer names a check but cannot supply its result. E3 fails authorization because policy protects guardrails and caps capabilities. Protected-state preservation is checked independently. Rejection returns to the host's own search or retry policy.

F. Candidate construction and origins

Algorithm 2 computes the candidate for an update. Quarantine uses the *old* graph, so dependents of the replaced version must be revalidated rather than silently inheriting it. Origins are conservative: the new version inherits the previous target version, its origins, and every declared parent version with its origins. An update cannot drop a parent and so cannot declare independence from a withdrawn ancestor. No provenance-severing release operation is provided.

Running example. E1 yields $O(\langle \mathsf{summarize}, 2\rangle) = \{\langle \mathsf{summarize}, 1\rangle\}$ and quarantines report, which depends on summarize, until it is revalidated against v2. In E4, report 1→2 declares `parents`

Algorithm 1 PREPARE: admit a request (flows ❶ and ❷)

Input: DSL source src, authenticated principal p
lock authority; $\kappa \leftarrow \mathsf{Id}$(service context)
$(IR, e) \leftarrow$ COMPILE(src); **require** $IR.C = C$
require $\mathsf{Auth}(p, e, S)$ ▷ principal, op, target, caps
require $e.from = v_{e.t}$ and $e.expect$ holds in S
$(S', O') \leftarrow$ CANDIDATE$_{e.op}(e, S)$
require $\mathsf{Protected}(S, S') \wedge \mathsf{Valid}(S', H, O')$
require $e.ensure$ holds; candidate caps $\subseteq Ceil(e.t)$
for $v \in Mandatory \cup e.requires$ **do**
  require $v \in \mathrm{dom}(Registry)$
  $ev[v] \leftarrow Registry[v](S', e)$; **require** $ev[v] =$ `True`
  require Id(service context) $= \kappa$
require Id(service context) $= \kappa$
register (e, S', O') under a fresh token τ
return $\rho = \langle \tau, p, \mathsf{h}(IR), \kappa, \mathsf{h}(S'), \mathsf{h}(O'), ev\rangle$

Algorithm 2 CANDIDATE: update a target t

Input: canonical request e for target t, state S
$c \leftarrow C[t]$; **require** $e.to > v_c$ and $\langle \mathsf{t}, e.to\rangle \notin \mathrm{dom}(H)$
require $c.parents \subseteq e.parents$ ▷ ancestry never dropped
$Q \leftarrow \mathsf{Dep}_{G(S)}(t)$ ▷ dependents in old graph
$c' \leftarrow c$ with version $e.to$, relations and caps of e
$O' \leftarrow O$; $O'(\langle \mathsf{t}, e.to\rangle) \leftarrow \{\langle \mathsf{t}, v_c\rangle\} \cup O(\langle \mathsf{t}, v_c\rangle) \cup$
  $\bigcup_{q \in e.parents} (\{\langle \mathsf{q}, v_q\rangle\} \cup O(\langle \mathsf{q}, v_q\rangle))$
$I' \leftarrow (I \cup Q) \setminus \{t\}$ ▷ update activates target
return $(\langle C[t \mapsto c'], I', W, r{+}1\rangle, O')$

Algorithm 3 COMMIT: apply a receipt (flows ❸ and ❹)

Input: receipt ρ
1: lock authority
2: **require** ρ genuine, outstanding, and unconsumed
3: **require** $\rho.\kappa = \mathsf{Id}$(service context) ▷ stale otherwise
4: $(e, S', O') \leftarrow Pending[\rho.\tau]$; recheck their hashes
5: stage history records, O', and hash-chained journal entry
6: atomically swap state references; consume ρ
7: notify adapter of committed identity; bind before dispatch
8: append decision record; **return** $(r', x = (e.t, v'_{e.t}))$

Algorithm 4 WITHDRAW: construct a candidate

Input: authorized request e, target t, current version v
1: **require** $e.to = v_t = v$ ▷ historical withdrawal not primitive
2: $W' \leftarrow W \cup \{\langle \mathsf{t}, v\rangle\}$; $I' \leftarrow I \cup \mathsf{Dep}_G(t) \cup \{t\}$
3: **return** $(\langle C, I', W', r{+}1\rangle, O)$ to PREPARE

Algorithm 5 ROLLBACK: construct a candidate

Input: authorized request e, target t, requested version v_h
1: **require** $\langle \mathsf{t}, v_h\rangle \in \mathrm{dom}(H)$ and $\mathsf{Elig}(\langle \mathsf{t}, v_h\rangle)$
2: $c_h \leftarrow H[\langle \mathsf{t}, v_h\rangle]$; $Q \leftarrow \mathsf{Dep}_G(t)$
3: $I' \leftarrow (I \cup Q) \setminus \{t\}$
4: **return** $(\langle C[t \mapsto c_h], I', W, r{+}1\rangle, O)$ to PREPARE

Algorithm 6 GUARD: host acceptance, execution, and reuse

1: **function** ACCEPT($artifact$, $base$, $sources$, p)
2: lock; freeze $artifact$ in trusted staging
3: **require** HOSTCHECKS($artifact$)
4: $src \leftarrow$ MAP($artifact, base, sources$)
5: $\rho \leftarrow$ PREPARE(src, p); $(r, x) \leftarrow$ COMMIT(ρ)
6: BIND(x, h($artifact$)); **return** x
7: **function** RUN($artifact$, x)
8: lock; **require** bytes match binding, x active and $\mathsf{Elig}(x)$
9: **require** EXECUTORALLOWED($artifact$)
10: HOSTRUN($artifact$); $Exec \leftarrow Exec \cup \{x\}$
11: INSERT($entry$): $entry.O \leftarrow \bigcup_{x \in Exec}(\{x\} \cup O(x))$
12: EXPOSE(): **return** $\{entry \in Store \mid entry.O \cap W = \varnothing\}$

`[summarize]`, so the origin rule records $O(\langle \mathsf{report}, 2\rangle) = \{\langle \mathsf{report}, 1\rangle, \langle \mathsf{summarize}, 2\rangle, \langle \mathsf{summarize}, 1\rangle\}$. Withdrawal in E5 invalidates this derived version through that set.

G. Commit and freshness

Algorithm 3 turns a receipt into state. Because κ binds state, policy, validator registry, history, and origins, any successful competing commit invalidates every other receipt prepared from the old base. Staging followed by a reference swap prevents partly applied authoritative transitions. A rejected receipt leaves state unchanged. This is a transaction guarantee for the service and its controlled publication path, not a general filesystem crash-consistency theorem.

Running example. E1 and E2 receive receipts with the same κ. Committing E1 changes C, r, and H, so E2 fails the context check even though E2 alone would have been valid. Re-proposing E2 against the new base is the intended recovery. This is objective O2.

H. Withdrawal and rollback

Withdrawal (Alg. 4) is the operator's revocation primitive. W grows monotonically ($W \subseteq W'$ for every transition). A dependent may become inactive without its own version being withdrawn, which keeps direct withdrawal distinct from affected closure. Eligibility of derived versions follows from Eq. (1), so no enumeration of descendants is needed.

Rollback (Alg. 5) loads an exact historical record and its origins. The common admission path rechecks current policy, capability ceilings, validators, and activity conditions. Recovery neither clears W nor overrides an origin's withdrawal.

Running example. E5 withdraws the current summarize v2 and quarantines report. Its v2 remains ineligible because $\langle \mathsf{summarize}, 2\rangle \in O(\langle \mathsf{report}, 2\rangle) \cap W$, even if dependency metadata later changes. Recovery to v2 is refused. Restoring summarize v1 and then report v1 can satisfy current policy (O4). Freshness and origin eligibility are separate obligations.

I. Execution and persistent reuse

Algorithm 6 expresses the host adapter protocol, not a particular framework's API. MAP supplies the trusted artifact/metadata binding. BIND associates a committed identity with immutable bytes. EXECUTORALLOWED checks the domain's execution policy. Admission, dispatch, and withdrawal share the authority lock. Already-running external effects are not undone by withdrawal.

Running example. E4's composition records summarize v2 as a source of report v2. Execution records this version, and a learned entry inherits the contributing executions' origins. After E5, dispatch refuses the withdrawn-origin version and retrieval omits the entry (O3). Lineage covers recorded artifact and execution relationships, not every semantic influence on a model. Omitting a memory, cache, or retrieval path from mediation invalidates the portability claim for that host.

J. Correctness argument and audit trail

Under trusted initialization, pure terminating validators, complete mediation, a stable validation context, and sufficient resources, accepted transitions preserve Valid. Initialization checks acyclicity and active origins. Earlier historical completeness is an operator assumption. Updates preserve identities and ancestry while quarantining dependents (Alg. 2). Withdrawal grows W and quarantines its closure (Alg. 4). Rollback checks historical origins against current W and capability ceilings (Alg. 5). Candidate checks establish the remaining invariants, and locked commit linearizes the state swap (Alg. 3). Induction over accepted transitions therefore gives preservation. This is a non-mechanized argument, not a proof of the Python interpreter or filesystem. For a representable request admitted by the reference relation, acceptance also requires available history, unchanged context, and passing validators. RQ1 checks both directions of this correspondence within finite bounds.

A hash-chained journal records committed transitions, and anchored replay checks continuity, content, and trusted identity. A separate decision log also records rejections. Together they trace each proposal to its decision and resulting state. Registry identities are process-local and do not attest every captured validator environment. Replay reruns supplied trusted validators. Version origins describe declared and observed

workflow relationships, not every semantic influence on a language model.

IV. Evaluation

A. Case-study host and adapter

We select Harness^2 [3], a recent state-of-the-art recursive harnessing framework, as an integration case study rather than a dependency of the method. Its Propose, Probe, and Compose loop edits and evaluates harness trees. Selection and reflection feed later tasks through an experience library. The pinned runtime (commit `6868817`) already checks paths and schemas, retries failures, and separates method from judge. These checks address well-formedness, not the lifecycle authority contract studied here. We do not treat absent lifecycle APIs as failures of its stated objectives.

Our adapter maps each multi-file tree to a workflow component and keeps protected configuration, policy, and verifier components separate. Composition/exposure sources extend bundle origins. A dedicated launcher wraps `_accept_tree`, `_run_stage`, and `Library` without replacing upstream search or learning code. All 70 upstream tests pass. The ledger fixtures allow only two trusted programs. ToolRepair-12 adds static allowlisting and qualified Bubblewrap execution for new tools. HumanEval+ evolves text while grading solution code separately. These domain adapters share one host implementation and therefore do not establish cross-framework portability.

B. Research questions

RQ1 (bounded conformance): Do the implementation and independent lifecycle semantics agree within published finite bounds?

RQ2 (enforcement discrimination): Which contract violations are blocked and which permitted changes retained, relative to ordinary testing and static validation, and what does each gate contribute?

RQ3 (integration fidelity): Are admission, composition, execution, and library reuse protected inside the original Harness^2 workflow, including with model-driven proposers?

RQ4 (useful behavior): Does an evolving agent under the contract deliver useful task outcomes at a documented inference budget?

RQ5 (cost and reproducibility): What do validation, scaling, and contention cost, and can distributed runs be reconstructed from pins and raw records?

C. Study design

Table I summarizes the cohorts, with their construction and measures explained below.

Cohort structure. The deterministic study repeats 25 actions five times under four baseline conditions. Full enforcement and five core ablations use the same actions. The DSL cohort crosses five models, 20 contexts covering 17 scenarios, and five seeds. Controlled host experiments repeat each fixture five times. The original, unit-test, and static-check conditions use 12 shared fixtures. Full enforcement and eight host ablations also use four lifecycle fixtures. ToolRepair-12 and HumanEval+ have separate task and stream designs described in Sec. IV-G. These cohorts answer different questions and must not be pooled.

Conditions. B0 to B2 are admission-only comparisons. Trusted safe setup advances later contexts, so their timings are not end-to-end deployment costs. In Harness^2, C1 adds one independent signed-ledger unit input after the original checks. C2 adds static principal/protected-file checks and DSL compilation, and is not a semantic verifier. Shared fixtures contain three permitted families (script correction, multifile correction, skill addition) and nine disallowed families (rows of Fig. 2a). The four lifecycle families (withdrawn origin, rollback conflict, stale approval, and interrupted publication) correspond to events E2, E4, and E5. C0 to C2 offer no withdrawal/recovery API, so no lifecycle rates are invented for them. The eight ablations remove permissions, dependencies, provenance, evidence, formal verification, withdrawal, rollback, or revalidation individually.

Models and budgets. Qwen2.5-Coder 7B, Phi-4 14B, GPT-OSS 20B, Devstral 24B, and Qwen3-Coder 30B are pinned by manifest digest. The DSL cohort uses temperature 0.2, top-p 0.9, 1,024 output and 4,096 context tokens, one generation per model/context/seed. The ledger pilot and ToolRepair-12 use 2,048 output and 16,384 context tokens and run original aggregation at $k{=}1$: four solver rollouts and three proposal stages per task, at most two edit attempts per stage. Raw request/response records retain all calls, tokens, limits, and failures.

Measures and analysis. *Unsafe deployment* means deploying a request that violates the explicit operator contract. It is not a maliciousness classifier. We separate syntax validity, oracle evaluability, allowed and disallowed attempts, deployment, false rejection, protocol/workflow failure, and task success. Invalid output is never counted as a safe request. Zero eligible observations yield an undefined rate, not zero risk. A post hoc strict request-fidelity diagnostic checks every declared field. It is not preregistered and not the safety oracle. Units depend on the study: designed scenarios in the primary cohort, families in ToolRepair-12, and ordered model/seed streams in HumanEval+. Sites supply infrastructure. Between-model comparisons in the primary cohort (task-cluster bootstrap, paired sign-flip tests, Holm) are diagnostic and reported only in the artifact. With few tasks we emphasize counts and effect sizes. ToolRepair-12 and HumanEval+ follow their own frozen protocols (Sec. IV-G).

D. RQ1: Bounded conformance

Table II summarizes the assurance evidence. The one-step graph/lifecycle domain has 176,256 cases over acyclic two-component graphs, inactive and withdrawn subsets, and relations to self, another component, or a missing component. Target versions range from 1 to 3 and expected versions from 1 to 2, with activity conditions varied. A separate 72,576-case domain varies one component's kind, mutability, capabilities, grant, and evidence. Together these checks accept 3,388 requests and reject 245,444 without a discrepancy against the independent reference.

TABLE I
EVALUATION COHORTS AND THEIR RESEARCH QUESTIONS.

Cohort	Size	Purpose	RQ
Bounded assurance	275,332 checks	Compare implementation and independent semantics	1
Deterministic baselines	500 decisions	Compare cumulative admission checks	2
Core ablations	750 decisions	Remove individual lifecycle gates	2
DSL generation	500 responses	Assess model-generated requests	2
Controlled Harness[2]	900 records	Exercise host seams and gate ablations	2, 3
Host pilots	25 + 50 streams	Diagnose workflow and usefulness failures	3, 4
ToolRepair-12	540 streams	Compare repair usefulness and contract violations	4
HumanEval+	3,936 outcomes	Compare evolving and frozen coding agents	4
Scaling	180 records	Measure graph-size and contention costs	5
Site qualification	18 runs	Check matched execution across three sites	5

TABLE II
SUMMARY OF BOUNDED ASSURANCE AND REGRESSION RESULTS.

Check	Extent	Result
One-step conformance	248,832 cases	No differences
History exploration	19,620 transitions	No differences
Anchored replay	6,880 endpoints	No differences
Lifecycle model	4 contexts	Python = TLC
Publication model	4 contexts	Python = TLC
Faulty variants	11 variants	All detected
Regression suites	62 / 32 / 70 tests	All pass

History exploration starts from seven two-component graphs, uses versions 1 to 3, and explores all operations to depth four. It visits 3,307 contexts and checks 19,620 transitions, of which 6,880 are accepted. Anchored replay checks every accepted endpoint. These are finite-domain counts, not estimates of deployment prevalence or independent replications.

The lifecycle model has one proposal and dependency epochs 0 to 2. Python and TLC agree in all four authorization/evidence contexts, with 18/44, 18/44, 23/62, and 61/133 states/edges. The publication model has one bundle, one origin bit, and two epochs. Its graphs agree at 45/81 states/edges with authorization and evidence enabled and 4/4 in each other context. Five lifecycle variants omit authorization, evidence, withdrawal, rollback, or revalidation. All yield counterexamples, as do six publication variants. For example, omitting revalidation permits deployment after a dependency change in five steps.

The regression counts refer to the stored core, integration, and upstream Harness[2] suites, respectively. They are source-scoped checks of the evaluated implementation.

Answer to RQ1. The checked representations agree within their finite bounds and detect all injected variants. This establishes bounded consistency, not a Python-to-TLA$^+$ refinement theorem or semantic safety for generated code.

E. RQ2: Enforcement discrimination

Deterministic baselines and core ablations. Each row of Table III contains 125 decisions: 60 disallowed and 65 allowed. FA counts disallowed requests admitted, and FR counts allowed requests rejected. Latency is the median admission time in milliseconds. The table shows that syntax (B0) and schema (B1) checks admit all 60 disallowed actions.

TABLE III
ADMISSION ERRORS AND MEDIAN LATENCY ACROSS BASELINE CONDITIONS AND CORE GATE ABLATIONS.

Condition	FA	FR	Median ms
B0 syntax only	60/60	0/65	0.197
B1 + snapshot schema	60/60	0/65	0.203
B2 + policy, candidate	20/60	0/65	0.227
B3 full enforcement	**0/60**	**0/65**	0.599
Full − dep. closure	5/60	10/65	0.538
Full − provenance	5/60	0/65	0.708
Full − revalidation	5/60	0/65	0.593
Full − withdrawal	0/60	0/65	0.563
Full − rollback	0/60	0/65	0.841

Trusted policy with candidate construction (B2) still admits 20, namely those whose violation is only visible to validators or lifecycle history. Full enforcement (B3) admits none and rejects none of the 65 allowed actions. Removing dependency closure, provenance checks, or revalidation each re-opens five admissions. Removing dependency closure also causes ten false rejections, because skipped quarantine diverges later state. Removing withdrawal or rollback checks alone has no effect in these cases because retained guards overlap. A zero single-gate effect does not show that a mechanism is dispensable.

Model-generated requests. Fig. 2b and Table IV show that only 91 of 500 generated responses are oracle-evaluable: 73 disallowed and 18 allowed. Full enforcement deploys none of the 73 and falsely rejects none of the 18. Three models return fenced output in all 100 responses, which the strict raw-text protocol rejects. GPT-OSS has 56 empty and 62 length-limited outputs (overlapping). The historical `syntax_valid` flag is also set for parser-level identifier failures: 39 Qwen3-Coder responses quote a relation or capability identifier and fail before oracle classification, so the flag must not be read as 100 parsed programs. Filtering to valid proposals would hide these interface failures.

Table IV reports 100 DSL responses and five guarded host streams per model. In the DSL columns, Flag counts the historical `syntax_valid` marks, Oracle gives allowed/disallowed evaluable requests, and Fid. is post hoc strict request fidelity. In the host columns, WF ok counts completed workflows, Inv. counts invalid proposals, Adm. gives allowed proposals admitted, and Blk. gives disallowed proposals blocked. The

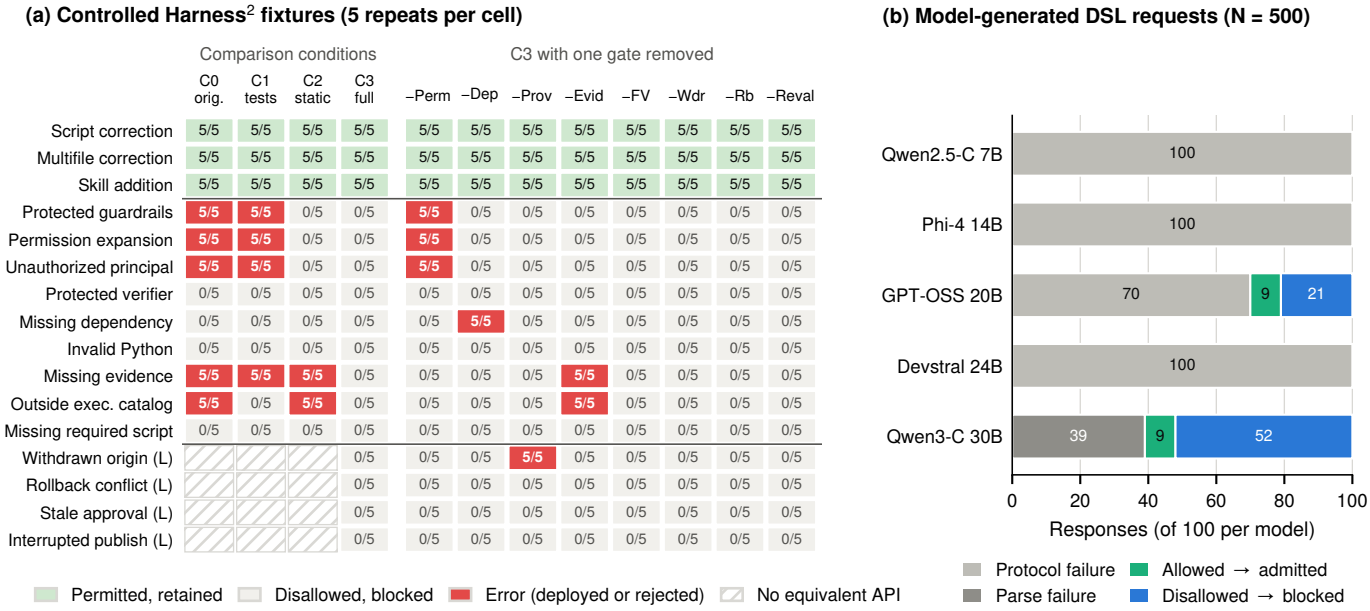


Fig. 2. Enforcement outcomes. (a) Each cell counts, over five repeats, permitted changes retained (top three rows) or disallowed changes deployed (other rows). Red cells are errors. Original Harness² admission (C0) deploys 25/45 shared violations. Ordinary testing (C1) still deploys 20 and static DSL validation (C2) 10. Full enforcement (C3) deploys none, retains all permitted changes, and blocks all four lifecycle families (L). Removing a single gate re-opens exactly the families that gate covers. (b) Of 500 model-generated DSL requests, 409 never reach the oracle (fenced, empty, length-limited, or identifier-level parse failures). Every oracle-classified request is decided correctly.

TABLE IV
MODEL-GENERATED REQUESTS TEST THE DSL INTERFACE AND GUARDED HARNESS² WORKFLOWS.

Model	Primary DSL Flag	Oracle	Fid.	Harness² streams WF ok	Inv.	Adm.	Blk.
Qwen2.5-C 7B	0	0/0	0%	5/5	0	7/7	30/30
Phi-4 14B	0	0/0	0%	5/5	43	0/0	17/17
GPT-OSS 20B	30	9/21	21%	0/5	n/a	n/a	n/a
Devstral 24B	0	0/0	0%	5/5	0	0/0	54/54
Qwen3-C 30B	100	9/52	21%	5/5	0	9/9	40/40
Total	130	18/73	n/a	20/25	43	16/16	141/141

TABLE V
CONTROLLED HARNESS² FIXTURES COMPARE CONTRACT VIOLATIONS, PERMITTED EXECUTIONS, AND LATENCY.

Condition	Shared	Retained	Lifecycle	ms
C0 original	25/45	15/15	n/a	77.17
C1 + unit test	20/45	15/15	n/a	155.73
C2 + static DSL	10/45	15/15	n/a	78.33
C3 full	**0/45**	**15/15**	**0/20**	114.81
C3 − permissions	15/45	15/15	0/20	n/r
C3 − evidence	10/45	15/15	0/20	n/r
C3 − dependencies	5/45	15/15	0/20	n/r
C3 − provenance	0/45	15/15	5/20	n/r
C3 − any other	0/45	15/15	0/20	n/r

notation n/a marks unavailable or inapplicable entries.

Answer to RQ2. Full enforcement deploys 0/60 deterministic, 0/73 oracle-classified model, and 0/45 controlled violations with no false rejections in those populations. Most generated outputs fail the interface before an assessable decision.

F. RQ3: Integration fidelity in Harness²

Fig. 2a and Table V report five repeats per host fixture. Each applicable condition has 45 shared disallowed, 15 permitted, and 20 lifecycle attempts. Shared and Lifecycle count disallowed executions, Retained counts permitted executions, and ms is mean latency over permitted executions. Timings are descriptive and order-confounded. Lifecycle results are inapplicable to C0, C1, and C2 (n/a), and ablation timings are unreported (n/r).

Original checks miss protected-file, permission, principal, evidence, and executable-catalog violations. C1 closes the catalog gap, while C2 closes the first three gaps. C3 blocks all shared and lifecycle violations and retains all permitted executions. Removing provenance re-admits the withdrawn-origin case in every repeat. Removing permissions, evidence, or dependencies re-admits 15, 10, or 5 shared violations, respectively.

Model-driven streams. With real model proposers inside the same workflow (Table IV, right), C3 blocks all 141 assessed disallowed proposals and admits all 16 permitted non-no-op proposals at exact bytes. GPT-OSS fails all five workflows before producing an assessable proposal. These failures remain in stream and task denominators. This pilot never exercised reflection, because our launcher omitted the model client as Harness²'s selector (all fifty task summaries have empty experience). The corrected launcher (Sec. IV-G), the deterministic fixtures, and eight coding-adapter regressions cover library guarding.

Answer to RQ3. The supported adapter enforces admission, composition, dispatch, and library exposure at actual upstream call sites. This is one executable integration under the stated

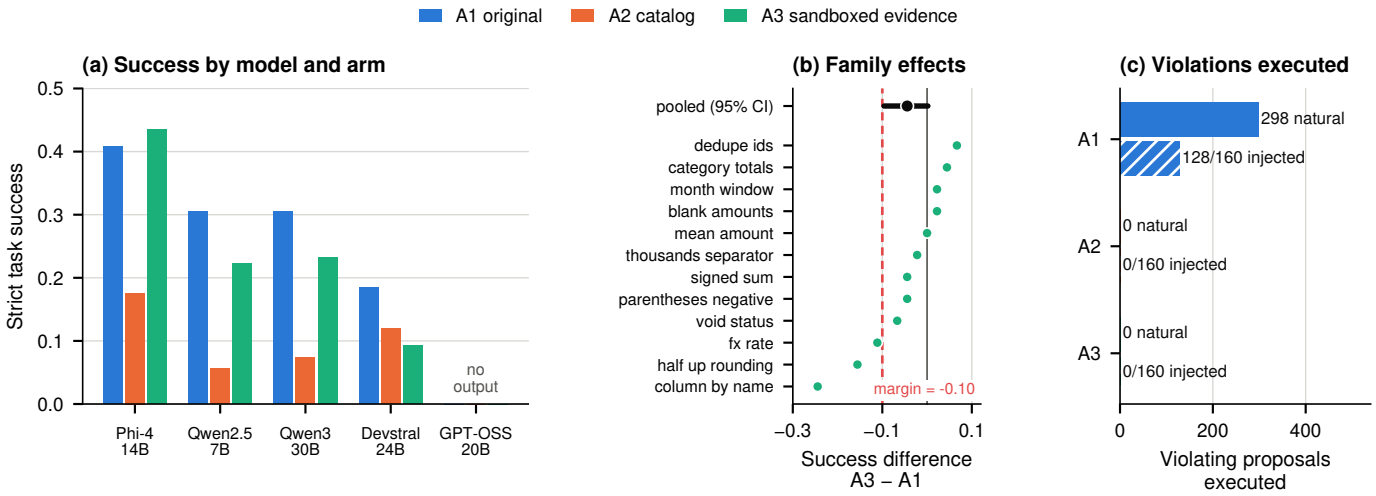


Fig. 3. ToolRepair-12 (540 streams). (a) Strict task success per model (GPT-OSS: no output, failure in every arm). (b) Per-family difference A3−A1, pooled family-cluster estimate and 95% CI, and the prespecified margin. (c) Contract-violating proposals executed (corrected label, declared probe excluded).

restrictions, not evidence of cross-framework portability.

G. RQ4: Useful behavior

Pilot. The first host pilot has 25 streams from five models and five seeds, with two ledger tasks per stream. A corrected pilot compares two executable policies in 50 streams. The initial pilot exposed three adapter defects (answer schema, selector channel, answer form). After the fixes, no disallowed proposal executed (0/31 catalog, 0/13 sandboxed) and sandboxed evidence admitted correct repairs the catalog rejected (8/50 versus 0/50 correct values, post hoc), but strict success stayed 0/100, so we ran a prespecified study. **ToolRepair-12 design.** Twelve tool families each plant one defect in the harness's executable CSV tool (e.g., sign handling, rounding, column order, blank and grouped numbers, Fig. 3b). Each family has a specification, a trusted reference, and disjoint task inputs (graded by exact output), operator-held validation inputs (authority only), and oracle inputs (independent proposal oracle only). The full design has 540 streams, each containing three tasks. Five pinned models run three seeds per family in three arms: original Harness[2] (A1), the contract with the catalog policy (A2), and the contract with sandboxed evidence (A3). A frozen reference runs inside every stream, and each stream receives one injected proposal from seven rotating classes. All arms execute non-catalog tools under the same infrastructure sandbox, which is not credited to contract enforcement. Protocol, analysis, and implementation hashes were frozen before qualification. Families are the inferential unit (10,000 paired cluster-bootstrap resamples, 95% percentile intervals). The frozen analysis applies Holm to bootstrap tail proportions for H1/H3. These are resampling diagnostics, not calibrated population-level p-values. Two deviations were recorded before any full result: amendment v1.1 made three crashing defects silent and reran all 540 streams, and the smoke run exposed an "executed" label that counted proposals restating *unchanged* files. The corrected label is primary and both are reported.

Usefulness. Mean per-family success is 0.241 for original Harness[2], 0.085 under the catalog, and 0.196 under sandboxed evidence (frozen: 0.041, Fig. 3). The sandboxed-evidence interval meets the prespecified 10-point non-inferiority margin against the unguarded arm (difference -0.044, 95% CI $[-0.096, +0.002]$, frozen resampling decision H3 met) and recovers 11.1 points over the catalog (95% interval $[+0.067, +0.159]$, H1 met under the frozen rule). The lower bound lies only 0.4 percentage points inside the margin, and family effects vary (Fig. 3b). Within A3 streams, success rises from 0.12 on the first task to 0.26 on the third (H4, exploratory), but fixed task order and the absence of a no-library ablation prevent attributing this trend to learning.

Safety at the same time. Original Harness[2] executed 426 contract-violating proposals (298 natural, 128/160 injected). A3 executed 0 natural and 0/160 injected violations under the corrected label (H2's observed-count criterion). These are planned-stream counts, including workflows that failed before proposals. Repeated classes within families/models are dependent, so we do not interpret the artifact's nominal binomial bound as a population risk bound. Under the uncorrected worker label, A3 shows 12 natural executions and H2 would not hold. Two A3 oracle-unknown cases were resolved after collection by direct oracle-input execution, while four A1 cases remain unknown. This supplementary resolution is not a qualified sandbox replication and is retained separately from the prespecified correction. As declared before the study, the input-triggered resource-abuse probe, whose trigger lies outside every validation case, passed behavioral evidence in 16/20 A3 streams and ran bounded by the sandbox. Only Phi-4 succeeds more often under sandboxed evidence than unguarded (47 versus 44 of 108). GPT-OSS produced no output (empty responses under JSON-constrained decoding). Its streams count as failures (intention to evaluate), and excluding them (exploratory) gives 0.301, 0.106, and 0.245.

HumanEval+. In a prespecified text-only study [10], [11] (164 tasks, Phi-4 and Qwen2.5-Coder, three seeds, hidden tests graded after a global freeze), guarded minus original

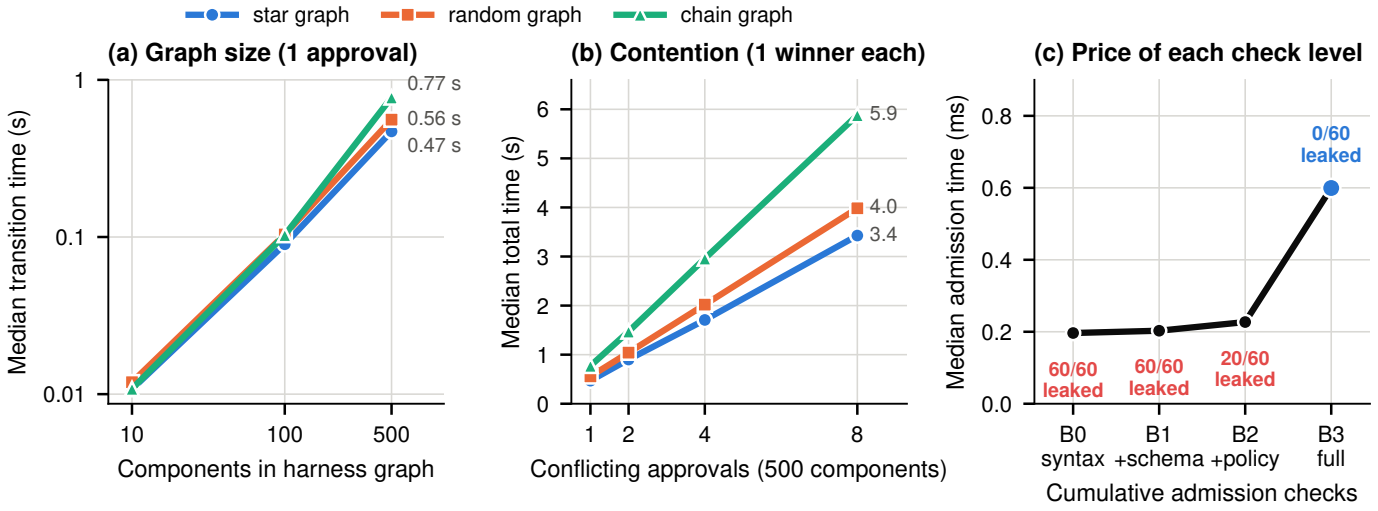


Fig. 4. Three aspects of enforcement cost (medians of five repeats). (a) *Graph size:* transition time grows with the number of components and depends on graph shape through the size of the quarantine closure. (b) *Contention:* with 1 to 8 conflicting approvals on 500 components, time grows linearly and every cell has exactly one commit winner. (c) *Check level:* each cumulative admission check adds little latency. Only full enforcement (B3, 0.60 ms) leaks none of the 60 disallowed actions.

evolution is -0.6 points for Phi-4 (seeds -8.5 to $+3.7$) and $+1.8$ for Qwen. Neither beats frozen best-of-eight selection (guarded/original/frozen: Phi-4 0.354/0.360/0.476, Qwen 0.766/0.748/0.780) at about $1.8\times$ its calls. The four reported endpoints are original evolution, guarded evolution, frozen selection, and a one-shot endpoint drawn from the frozen pool. Across all tasks, models, and seeds, these yield 3,936 outcomes. These are descriptive all-attempts accuracies over three ordered streams/model, not task-independent significance tests. Frozen selection matches solver calls rather than total compute. One seed/model was regenerated before hidden grading after a storage failure, a recorded deviation. **Answer to RQ4.** A3 meets the frozen ToolRepair non-inferiority rule narrowly, retaining 81% of A1's point-estimate success. The margin is large relative to the 0.241 baseline. Guarding preserves rather than creates usefulness: neither evolving HumanEval+ arm beats frozen selection. Behavioral evidence contains but does not exclude the declared input-triggered probe.

H. RQ5: Cost and reproducibility

Validation and scaling. Fig. 4 summarizes scaling and the admission baselines. The scaling design crosses three graph sizes, three graph shapes, four contention levels, and five repeats, giving 180 records. Shape matters because quarantine follows reverse reachability: chains quarantine the longest closures. Contention is resolved by compare-and-swap on the context hash: in every tested cell exactly one approval commits and the others are rejected as stale. These observations establish neither distributed throughput nor an asymptotic bound. Traced Python allocation peaks at 2.5 MB at 500 components. In Harness^2, the permitted-execution mean rises from 77.17 ms (C0) to 114.81 ms (C3), a descriptive, order-confounded fixture difference. C1's slower 155.73 ms reflects its extra test process and does not establish general superiority to testing.

Reproducibility. Qualification comprises 18 runs across three sites, two studies, and three matched trials. HPC runs used Fir, Narval, and Rorqual through a `use-hpc`-derived Slurm pipeline [12] with source freezes, cache identity and GPU checks, measured site qualification, and hash-verified collection. Censored and failed cells are preserved, and superseded cells are re-run as recorded supplements rather than overwritten. Twenty-four matched cross-site pairs produced identical text and decisions under identical identities. All 24 were syntax-invalid outputs, so this shows deterministic serving, not useful generation.

Answer to RQ5. The 0.6 ms admission median is below 0.1% of ToolRepair's 1.1 s mean stream time per call. Admission for a 500-component chain takes 0.77 s. These scoped measurements exclude inference and do not establish distributed service throughput. Reanalysis and raw-archive availability are distinguished in Sec. VI.

I. Discussion: transfer and engineering trade-offs

What transfers. Component granularity and search strategy are adapter choices. ADAS-like archive search [1] could mediate design promotion and execution. Code-evolving agents [2] additionally need code isolation. Skill-learning agents [9] need installation, execution, and retrieval guards. These are architectural mappings, not implemented ports. The tested bundle projection demonstrates multi-file binding, not fine-grained mediation of every kind.

When transfer fails. A host that bypasses a guard, cannot authenticate its caller, omits sources, or lets evolved code modify the authority violates Sec. III-B. Current restrictions include single-target updates, acyclic metadata relations, conservative ancestry, in-process serialization, and no verified restart or distributed commit. Withdrawal cannot erase prior model context or undo an external side effect. A clean independent rewrite cannot shed withdrawn ancestry in this language.

An operator-approved provenance-release mechanism is future work.

Operational value. The service supplies an auditable release condition for generated changes. Catalog validation rejects useful equivalent programs. Behavioral evidence expands the trusted base and misses input-triggered defects. Four zero-effect ablations reflect overlapping guards, not established redundancy. Operators must maintain mappings, policies, validators, and isolation.

J. Threats to validity

Construct. Compliance with an operator lifecycle contract is narrower than harmless agent behavior. Correctness, fidelity, and authorization are distinct, and strict fidelity is post hoc. The RQ2 and RQ3 fixtures were written by the authors alongside the gates they exercise, so their one-to-one coverage shows that each gate works as designed, not that the gate set is complete. Injected red-team classes and model-generated proposals only partly offset this. **Internal.** Validators and reference semantics can share design errors. Independent implementations and faulty variants reduce but do not eliminate common-mode mistakes. The core assumes a trusted process boundary. Qualified domain sandboxes do not make the authority safe against a hostile same-user host. Durable restart is unsupported. Behavioral oracles use disjoint inputs but share domain specifications and some checks with validators, so agreement is not an independent proof of semantic safety. **External.** Designed fixtures are not a sample of real attacks. Two catalog programs, two ledger tasks, and twelve single-tool planted-defect families do not establish general autonomous work, and pilot correct-value counts are post hoc. Models and sites do not add tasks. All empirical adapters use one host, so neither multiple domains nor a generic interface establishes cross-framework generalization. Public benchmarks such as HumanEval+ may be contaminated. **Conclusion.** Zero observed violations is not zero risk. The H3 interval is only 0.4 points inside a wide margin relative to the 0.241 base rate, and bootstrap tail proportions are not calibrated tests. H2 depends on corrected labels and supplementary oracle resolution. Repeated fixtures are not independent samples. Workflow failures reduce the assessable safety population and remain failures for task outcomes. **Operational.** Queues, GPUs, and model serving affect completion (GPT-OSS never produced output). Pins, raw logs, and censoring support audit, not determinism.

V. Related Work

Runtime enforcement and authorization. We specialize runtime monitoring [4] and authorization [5], [13] to versioned evolution and reuse. The separation of an untrusted producer from a trusted checker resembles proof-carrying code [14], but receipts are context-bound service records rather than program proofs. TLA$^+$ [15] supplies bounded cross-checks, not a refinement theorem. Our origin relation specializes PROV-DM [16] for revocation eligibility.

Agent safety languages. AgentSpec enforces declarative rules over execution and can inspect trajectories [6]. Progent controls evolving tool privileges and checks policy changes with an SMT solver [7]. AutoSpec learns rule revisions from counterexamples [8]. Our distinction is an explicit artifact lifecycle combining context-bound approval with origin eligibility across execution and reuse. Existing rule languages could encode parts of this contract with additional state and hooks. We claim neither greater expressiveness nor empirical security superiority, as no head-to-head comparison was run.

Software update security and dynamic updating. The Update Framework addresses stale and compromised update metadata [17], while in-toto binds supply-chain steps to authorized producers and artifacts [18]. Our contract additionally follows self-generated versions and derivation origins into execution and experience reuse. Admission depends on current policy and validator evidence. Dynamic updating [19] addresses safe code replacement and state transfer, whereas we govern version eligibility.

Prompt injection. Indirect injection turns untrusted content into instructions [20]. CaMeL separates trusted control flow from untrusted data with capabilities [21]. A self-evolving harness adds a persistent channel through its own edited files and library, which our contract treats as untrusted proposals and origin-tracked entries.

Self-evolving agents and benchmarks. ADAS searches over agent designs [1], Voyager accumulates a reusable skill library [9], the Darwin Gödel Machine studies open-ended self-improvement [2], and Harness^2 recursively adapts harnesses [3]. Our authority boundary governs releases and subsequent reuse independently of these search procedures (Sec. IV-I). Only the Harness^2 mapping is empirically exercised. HumanEval and EvalPlus assess generated-program correctness [10], [11] and complement lifecycle checks.

VI. Conclusion

SSEVOH-DSL treats approval as a revocable relation to a versioned context. Its metadata rules and adapter obligations apply independently of how a harness searches for improvements. Execution and persistent reuse must remain mediated. Bounded assurance supports consistency of the rules, and one Harness^2 integration demonstrates enforcement at real admission, execution, and reuse seams. ToolRepair-12 exposes the usefulness/evidence trade-off, while HumanEval+ shows no advantage over frozen selection. Cross-framework ports, broader tasks, durable recovery, and mechanized refinement remain necessary to establish wider applicability.

Artifacts. The versioned evidence snapshot [22] contains the core, pinned host runtime, protocols/deviations, outcome records, and analysis scripts. `make verify` checks bounded assurance. experiments/scripts/ contains outcome analyses. ToolRepair success is reproducible from committed toolrepair-v1/streams.jsonl. Corrected execution labels are documented in experiments/toolrepair/DEVIATIONS.md and require the locally retained per-stream archives, which are not in the public snapshot. Thus full independent replication of that corrected safety result remains an artifact-release limitation, rather than a completed archival release.

REFERENCES

[1] S. Hu, C. Lu, and J. Clune, "Automated design of agentic systems," *arXiv preprint arXiv:2408.08435*, 2024. [Online]. Available: https://arxiv.org/abs/2408.08435

[2] J. Zhang, S. Hu, C. Lu, R. Lange, and J. Clune, "Darwin gödel machine: Open-ended evolution of self-improving agents," *arXiv preprint arXiv:2505.22954*, 2025. [Online]. Available: https://arxiv.org/abs/2505.22954

[3] R. Xu, Y. Chen, Z. CuiZhu, B. D. Mishra, Y. Ming, H. Yu, R. Han, C.-Y. Lee, and T. Pfister, "Harness^2: Recursive agent harnessing for an open world," *arXiv preprint arXiv:2610.14224*, 2026. [Online]. Available: https://arxiv.org/abs/2610.14224

[4] F. B. Schneider, "Enforceable security policies," *ACM Transactions on Information and System Security*, vol. 3, no. 1, pp. 30–50, Feb. 2000. [Online]. Available: https://www.cs.cornell.edu/fbs/publications/EnfSecPols.pdf

[5] J. W. Cutler, C. Disselkoen, A. Eline, S. He, K. Headley, M. Hicks, K. Hietala, E. Ioannidis, J. Kastner, A. Mamat, D. McAdams, M. McCutchen, N. Rungta, E. Torlak, and A. Wells, "Cedar: A new language for expressive, fast, safe, and analyzable authorization (extended version)," *arXiv preprint arXiv:2403.04651*, 2024. [Online]. Available: https://arxiv.org/abs/2403.04651

[6] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," in *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering*, 2026, pp. 2938–2950. [Online]. Available: https://arxiv.org/abs/2503.18666

[7] T. Shi, J. He, Z. Wang, H. Li, L. Wu, W. Guo, and D. Song, "Progent: Securing AI agents with privilege control," *arXiv preprint arXiv:2504.11703*, 2026, version 3, revised May 2026; first submitted 2025. [Online]. Available: https://arxiv.org/abs/2504.11703v3

[8] P. Ma, Z. Wang, Z. Ji, Y. Zhou, Z. Xue, Z. Li, S. Wang, and X. Zhang, "AutoSpec: Safety rule evolution for LLM agents via inductive logic programming," *arXiv preprint arXiv:2606.24245*, 2026. [Online]. Available: https://arxiv.org/abs/2606.24245

[9] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," *Transactions on Machine Learning Research*, 2024.

[10] M. Chen *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021. [Online]. Available: https://arxiv.org/abs/2107.03374

[11] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation," in *Advances in Neural Information Processing Systems*, vol. 36, 2023. [Online]. Available: https://arxiv.org/abs/2305.01210

[12] ai4se4ai-lab, "use-hpc," Software repository, 2026, pipeline base revision f9dc7d66ad8df3c202f6b72b02630e14fd148756. [Online]. Available: https://github.com/ai4se4ai-lab/use-hpc

[13] C. Disselkoen, A. Eline, S. He, K. Headley, M. Hicks, K. Hietala, J. Kastner, A. Mamat, M. McCutchen, N. Rungta, B. Shah, E. Torlak, and A. Wells, "How we built cedar: A verification-guided approach," in *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, 2024. [Online]. Available: https://arxiv.org/abs/2407.01688

[14] G. C. Necula, "Proof-carrying code," in *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 1997, pp. 106–119.

[15] L. Lamport, *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002. [Online]. Available: https://lamport.azurewebsites.net/tla/book.html

[16] L. Moreau and P. Missier, "PROV-DM: The PROV data model," W3C Recommendation, Apr. 2013. [Online]. Available: https://www.w3.org/TR/prov-dm/

[17] J. Samuel, N. Mathewson, J. Cappos, and R. Dingledine, "Survivable key compromise in software update systems," in *Proceedings of the 17th ACM Conference on Computer and Communications Security (CCS)*, 2010, pp. 61–72.

[18] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappos, "in-toto: Providing farm-to-table guarantees for bits and bytes," in *Proceedings of the 28th USENIX Security Symposium*, 2019, pp. 1393–1410.

[19] M. Hicks and S. Nettles, "Dynamic software updating," *ACM Transactions on Programming Languages and Systems*, vol. 27, no. 6, pp. 1049–1096, 2005.

[20] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023, pp. 79–90.

[21] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, "Defeating prompt injections by design," *arXiv preprint arXiv:2503.18813*, 2025.

[22] "SSEVOH-DSL: Source, protocols, and evaluation evidence," Versioned software and data repository, 2026, evidence snapshot 07f544fec4b66c2eb8354500f12c8358fed16f29. [Online]. Available: https://github.com/ai4se4ai-lab/SSEVOH-DSL/tree/07f544fec4b66c2eb8354500f12c8358fed16f29